

Aufgabe 3: Cache-Verhalten

1. Mögliche Cache-Konfigurationen (32 Words)

Die Gesamtgröße des Caches beträgt 32 Words. In allen Fällen werden die Strategien Write-back und Write-allocate verwendet.

Konfiguration 1:

- 32 Lines
- 1 Word pro Line
- Direct-mapped

Diese Konfiguration nutzt hauptsächlich temporale Lokalität. Räumliche Lokalität wird kaum ausgenutzt. Konflikt-Misses treten häufig auf.

Konfiguration 2:

- 8 Lines
- 4 Words pro Line
- Direct-mapped

Diese Konfiguration stellt einen guten Kompromiss dar. Sowohl temporale als auch räumliche Lokalität werden genutzt. Die Anzahl der Konflikt-Misses ist moderat.

Konfiguration 3:

- 2 Lines
- 16 Words pro Line
- Direct-mapped

Diese Konfiguration nutzt räumliche Lokalität sehr gut. Allerdings ist die Gefahr von Konflikt-Misses hoch, da nur wenige Cache-Lines vorhanden sind.

—

2. Wann ist Write-allocate nicht sinnvoll?

Write-allocate ist nicht sinnvoll, wenn Daten nur geschrieben, aber nicht erneut gelesen werden.

In solchen Fällen führt Write-allocate dazu, dass unnötige Cache-Lines aus dem Hauptspeicher geladen werden.

Typische Beispiele sind:

- Logging
- Streaming-Ausgaben
- Initialisierung großer Speicherbereiche

Hier ist No Write-allocate effizienter.

—

3. Vergleich der Write-Strategien bei Offset 64 und 128

Der verwendete Cache ist:

- Direct-mapped
- 8 Lines
- 4 Words pro Line

Der verwendete Datencache ist direct-mapped und besteht aus 8 Cache-Lines mit jeweils 4 Words pro Line.

Bei einem Offset von 64 Byte liegen die Quell- und Zieladressen in unterschiedlichen Cache-Lines. Daher treten nur wenige Konflikt-Misses auf.

Bei Verwendung von Write-back und Write-allocate ist die Hit-Rate in diesem Fall gut, da sowohl Lese- als auch Schreibzugriffe von der räumlichen Lokalität profitieren.

Bei Write-through und Write-allocate bleibt die Hit-Rate ebenfalls akzeptabel, jedoch verursacht jeder Schreibzugriff einen zusätzlichen Zugriff auf den Hauptspeicher.

Bei Write-back und No Write-allocate werden Schreibzugriffe nicht im Cache gespeichert. Die Lesezugriffe auf die Quelldaten profitieren weiterhin vom Cache, sodass die Hit-Rate insgesamt mittel ist.

Bei Write-through und No Write-allocate führen alle Schreibzugriffe direkt zu Speicherzugriffen. Dies verschlechtert die Performance, auch wenn keine starken Konflikt-Misses auftreten.

—

Bei einem Offset von 128 Byte entspricht der Abstand zwischen Quell- und Zieladressen der Gesamtgröße des Caches. In einem direct-mapped Cache werden beide Adressen auf dieselbe Cache-Line abgebildet.

Bei Write-back und Write-allocate führt dies dazu, dass sich Quell- und Zieldaten bei nahezu jedem Zugriff gegenseitig aus dem Cache verdrängen. Die Hit-Rate ist daher sehr schlecht.

Bei Write-through und Write-allocate tritt die gleiche Konfliktproblematik auf. Zusätzlich verursachen die Schreibzugriffe einen hohen Speicherverkehr.

Bei Write-back und No Write-allocate werden Zieladressen bei einem Write-Miss nicht in den Cache geladen. Dadurch bleiben die Quelldaten im Cache erhalten, und die Lesezugriffe haben eine gute Hit-Rate.

Auch bei Write-through und No Write-allocate bleiben die Quelldaten im Cache, da Schreibzugriffe den Cache umgehen. In diesem Fall ist die Hit-Rate ebenfalls gut.

Messergebnisse aus der Simulation (Ripes)

Die folgenden Messergebnisse wurden mit dem Simulator Ripes ermittelt. Es wurde ein gleitender Mittelwert über 50 Zyklen verwendet. Die Cache-Konfiguration entspricht der Aufgabenstellung.

—

Offset = 64 Byte

64	Hits	Misses
WB + WA	16	2
WT + WA	16	2
WB + NoWA	9	9
WT + NoWA	9	9

Table 1: Messergebnisse (Offset 64 Byte)

—

Offset = 128 Byte

128	Hits	Misses
WB + WA	2	16
WT + WA	2	16
WB + NoWA	9	9
WT + NoWA	9	9

Table 2: Messergebnisse (Offset 128 Byte)

Die Messergebnisse bestätigen die theoretische Analyse. Bei Offset 128 führen Write-allocate-Strategien zu starken Konflikt-Misses, da Quell- und Zieladressen auf dieselbe Cache-Line abgebildet werden. No Write-allocate verhindert diese Verdrängung und verbessert die Hit-Rate deutlich.

—

4. Besondere Problematik bei Offset 128 und Write-allocate

Bei Offset 128 entspricht der Abstand zwischen Quell- und Zieladresse genau der Größe des gesamten Caches.

In einem direct-mapped Cache führt dies dazu, dass beide Adressen auf dieselbe Cache-Line abgebildet werden.

Bei Verwendung von Write-allocate verdrängen sich Quell- und Zieldaten gegenseitig aus dem Cache. Dies führt zu einer sehr niedrigen Hit-Rate.

—

5. Modifikation des Programms zur Verbesserung der Hit-Rate

Eine Verbesserung der Hit-Rate kann durch eine gezielte Änderung des Programms erreicht werden.

Eine einfache Möglichkeit besteht darin, den Offset so zu verändern, dass Quell- und Zieladressen nicht mehr auf dieselbe Cache-Line abgebildet werden.

Beispielsweise kann der Offset von 128 auf 64 geändert werden.